



北京師範大學
BEIJING NORMAL UNIVERSITY

计算机图形学 实验报告

姓名 杨博文 学号 202311081015
院系 人工智能学院 专业 计算机科学与技术

2025 年 4 月 26 日

摘 要

计算机图形学的第五次实验，主要内容包括为 Assimp、网格、模型、光照，基于 OpenGL。

目录

1	运行环境及运行方式	3
2	任务 1: 展示全貌	3
2.1	任务要求	3
2.2	关键代码	3
2.3	结果图片	3
3	任务 2: 三种渲染效果	4
3.1	任务要求	4
3.2	关键代码	4
3.3	结果图片	5
4	任务 3: 增加点光源	7
4.1	任务要求	7
4.2	关键代码	7
4.3	结果图片	12
5	题外话	13

1 运行环境及运行方式

硬件为 Macbook Air, M2 芯片, 操作系统为 MacOS Sequoia 15.0。

IDE 为 Xcode Version16.0。

使用 GLFW+GLAD, 程序在 Xcode 内编译运行。

2 任务 1: 展示全貌

2.1 任务要求

在 “model_loading.cpp” 中修改相机视角或模型位置, 使得在初始视图中可以观察到完整模型

2.2 关键代码

```
Camera camera(glm::vec3(0.0f, 0.0f, 7.0f));  
...  
model = glm::scale(model, glm::vec3(0.8f, 0.8f, 0.8f));
```

第一行代码帮助我们修改相机位置, 离模型更远以看到全貌。第二行代码将模型大小缩小至原来的 0.8 倍, 使得全貌可以被看见。

2.3 结果图片

以上两个操作都做了之后, 让我们看一看最终的结果。

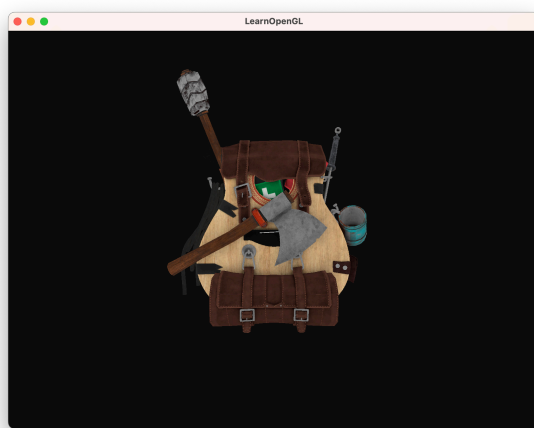


图 1: 实验 1 结果

3 任务 2: 三种渲染效果

3.1 任务要求

基于任务 1, 修改片段着色器代码, 展示仅有漫反射材质、仅有镜面材质、二者均有的三种不同渲染效果;

3.2 关键代码

```
1.model_loading.fs:
#version 330 core
out vec4 FragColor;

in vec2 TexCoords;

uniform sampler2D texture_diffuse1;
uniform sampler2D texture_specular1;

uniform int renderMode;

void main()
{
    vec3 diffuseColor = texture(texture_diffuse1, TexCoords).rgb;
    vec3 specularColor = texture(texture_specular1, TexCoords).rrr;
    vec3 result;

    if (renderMode == 1)
    {
        // 仅漫反射
        result = diffuseColor;
    }
    else if (renderMode == 2)
    {
        // 仅镜面反射
        result = specularColor;
    }
    else
    {

```

```
        // 默认显示漫反射 + 镜面反射
        result = diffuseColor + specularColor;
    }

    FragColor = vec4(result, 1.0);
}

main.cpp:
int mode=1;
...
    ourShader.setInt("renderMode",mode);
...
void processInput(GLFWwindow *window)
{
    ...
    if (glfwGetKey(window, GLFW_KEY_1) == GLFW_PRESS)
        mode = 1; // 仅漫反射
    if (glfwGetKey(window, GLFW_KEY_2) == GLFW_PRESS)
        mode = 2; // 仅镜面反射
    if (glfwGetKey(window, GLFW_KEY_3) == GLFW_PRESS)
        mode = 3; // 漫反射 + 镜面反射
}
```

我们对于着色器进行编辑，使得接受一个输入的模式数，然后进行判断，分别进行输出。然后对于主过程，增添键盘读取 1、2、3 切换对应模式的功能。

3.3 结果图片

我们对于一次运行，默认为仅漫反射的情况，如[图 2](#)。当我们键盘输入 2 时，切换为仅镜面反射的情况，即[图 3](#)。当我们键盘输入 3 时，切换为模式 3，即漫反射 + 镜面反射，如[图 4](#)。这里我们还转到了背面看模式 3 的背面，如[图 5](#)。

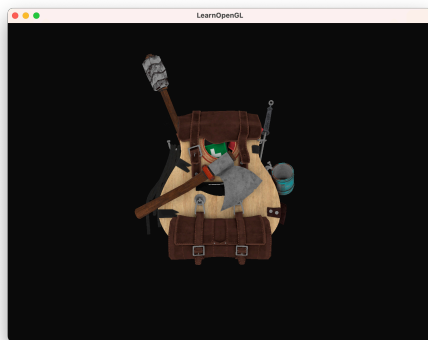


图 2: 实验 2 模式 1 结果

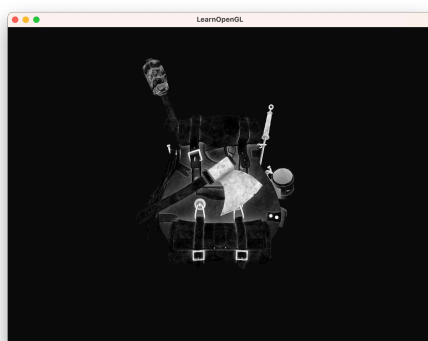


图 3: 实验 2 模式 2 结果

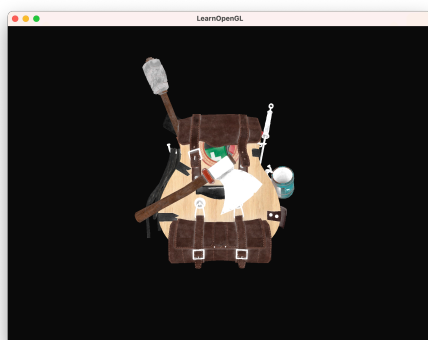


图 4: 实验 2 模式 3 正面结果

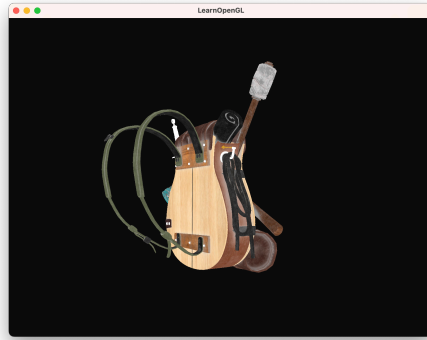


图 5: 实验 2 模式 3 背面结果

4 任务 3: 增加点光源

4.1 任务要求

结合“light_casters_point.cpp”，基于任务 2 增加点光源，并修改光源配置，展示仅有漫反射材质、仅有镜面材质、二者均有的三种不同渲染效果。

4.2 关键代码

```
5.2.light_casters.fs:
#version 330 core
out vec4 FragColor;

struct Material {
    float shininess;
};

uniform sampler2D texture_diffuse1;
uniform sampler2D texture_specular1;
...
void main()
{
    // ambient
    vec3 ambient = light.ambient * texture(texture_diffuse1, TexCoords).rgb;

    // diffuse
    ...
    vec3 diffuse = light.diffuse * diff * texture(texture_diffuse1, TexCoords).rgb;
```

```
    // specular
    ...
    vec3 specular = light.specular * spec * texture(texture_specular1, TexCoords).rrr
    ...
}
```

main.cpp:

```
// lighting
glm::vec3 lightPos(1.2f, 1.0f, 2.0f);
Shader lightingShader("/Users/yangbowen/Desktop/本科/大二/大二下课程
/计算机图形学/LearnOpenGL-master/src/2.lighting/5.2.light_casters_point
/5.2.light_casters.vs", "/Users/yangbowen/Desktop/本科/大二/大二下课程
/计算机图形学/LearnOpenGL-master/src/2.lighting/5.2.light_casters_point/
5.2.light_casters.fs");
    float vertices[] =
{
    -0.5f, -0.5f, -0.5f,
    -0.5f, 0.5f, -0.5f,
    -0.5f, -0.5f, 0.5f,
    -0.5f, 0.5f, 0.5f,
    0.5f, -0.5f, -0.5f,
    0.5f, 0.5f, -0.5f,
    0.5f, -0.5f, 0.5f,
    0.5f, 0.5f, 0.5f,
};
unsigned int indices[] =
{
    0,4,5,
    5,1,0,
    2,6,7,
    3,1,0,
    0,2,3,
    7,5,4,
    4,6,7,
    0,4,6,
    6,2,0,
```

```
        1,5,7,
        7,3,1
    };

    unsigned int VBO,lightCubeVAO,EBO;
    glGenVertexArrays(1,&lightCubeVAO);
    glGenBuffers(1,&VBO);
    glGenBuffers(1,&EBO);

    glBindVertexArray(lightCubeVAO);
    glBindBuffer(GL_ARRAY_BUFFER,VBO);
    glBufferData(GL_ARRAY_BUFFER,sizeof(vertices),
vertices,GL_STATIC_DRAW);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER,EBO);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER,sizeof(indices),
indices,GL_STATIC_DRAW);
    glVertexAttribPointer(0,3,GL_FLOAT,GL_FALSE,3*sizeof(float),(void*)0);
    glEnableVertexAttribArray(0);

    // view/projection transformations
    glm::mat4 projection = glm::perspective(glm::radians(camera.Zoom),
(float)SCR_WIDTH / (float)SCR_HEIGHT, 0.1f, 100.0f);
    glm::mat4 view = camera.GetViewMatrix();
    glm::mat4 model = glm::mat4(1.0f);

    model = glm::translate(model,lightPos);
    model = glm::scale(model,glm::vec3(0.2f));

    // be sure to activate shader when setting uniforms/drawing objects
    lightingShader.use();
    lightingShader.setVec3("light.position", lightPos);
    lightingShader.setVec3("viewPos", camera.Position);

    // light properties
/*    lightingShader.setVec3("light.ambient", 0.2f, 0.2f, 0.2f);
    lightingShader.setVec3("light.diffuse", 0.5f, 0.5f, 0.5f);
    lightingShader.setVec3("light.specular", 1.0f, 1.0f, 1.0f);
    lightingShader.setFloat("light.constant", 1.0f);
```

```
lightingShader.setFloat("light.linear", 0.09f);
lightingShader.setFloat("light.quadratic", 0.032f);

// material properties
lightingShader.setFloat("material.shininess", 32.0f);*/
//仅含漫反射
if(mode==1)
{
    lightingShader.setVec3("light.ambient", 0.0f, 0.0f, 0.0f);
    lightingShader.setVec3("light.diffuse", 1.0f, 1.0f, 1.0f);
    lightingShader.setVec3("light.specular", 0.0f, 0.0f, 0.0f);
    lightingShader.setFloat("light.constant", 1.0f);
    lightingShader.setFloat("light.linear", 0.0f);
    lightingShader.setFloat("light.quadratic", 0.0f);

    // material properties
    lightingShader.setFloat("material.shininess", 100.0f);
}
//仅含镜面反射
else if(mode==2)
{
    lightingShader.setVec3("light.ambient", 0.0f, 0.0f, 0.0f);
    lightingShader.setVec3("light.diffuse", 0.0f, 0.0f, 0.0f);
    lightingShader.setVec3("light.specular", 1.0f, 1.0f, 1.0f);
    lightingShader.setFloat("light.constant", 1.0f);
    lightingShader.setFloat("light.linear", 0.0f);
    lightingShader.setFloat("light.quadratic", 0.0f);

    // material properties
    lightingShader.setFloat("material.shininess", 100.0f);
}
else
{
    lightingShader.setVec3("light.ambient", 0.0f, 0.0f, 0.0f);
    lightingShader.setVec3("light.diffuse", 1.0f, 1.0f, 1.0f);
    lightingShader.setVec3("light.specular", 1.0f, 1.0f, 1.0f);
    lightingShader.setFloat("light.constant", 1.0f);
```

```
        lightingShader.setFloat("light.linear", 0.0f);
        lightingShader.setFloat("light.quadratic", 0.0f);

        // material properties
        lightingShader.setFloat("material.shininess", 100.0f);
    }

    lightingShader.setMat4("projection", projection);
    lightingShader.setMat4("view", view);

    model = glm::mat4(1.0f);
    model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
    model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
    lightingShader.setMat4("model", model);

    ourModel.Draw(lightingShader);

    lightCubeShader.use();
    lightCubeShader.setMat4("projection", projection);
    lightCubeShader.setMat4("view", view);
    model = glm::mat4(1.0f);
    model = glm::translate(model, lightPos);
    model = glm::scale(model, glm::vec3(0.2f)); // a smaller cube
    lightCubeShader.setMat4("model", model);

    glBindVertexArray(lightCubeVAO);
    glDrawElements(GL_TRIANGLES, 36, GL_UNSIGNED_INT, 0);
    glDrawArrays(GL_TRIANGLES, 0, 36);

    glDeleteVertexArrays(1, &lightCubeVAO);
    glDeleteBuffers(1, &VBO);
```

代码较长，主要内容包括：在着色器中改变纹理来源，定义灯光点位置，使用着色器，定义发光方块的坐标，设置发光方块等。

4.3 结果图片

运行结果如下，我发现，在以点光源为光源时，本来存在于背面的光（图 5）几乎接近看不见，如图 9，符合常理。

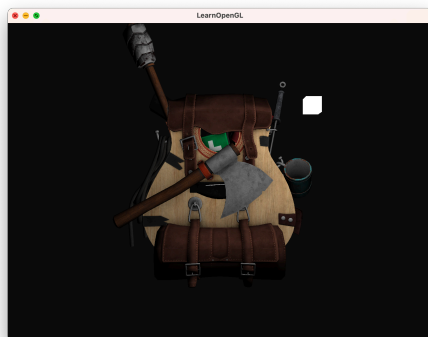


图 6: 实验 3 模式 1 结果

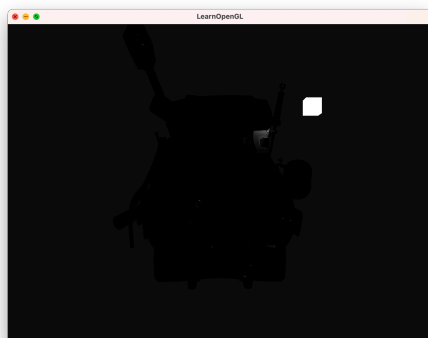


图 7: 实验 3 模式 2 结果

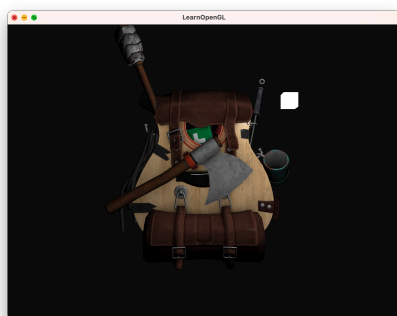


图 8: 实验 3 模式 3 正面结果

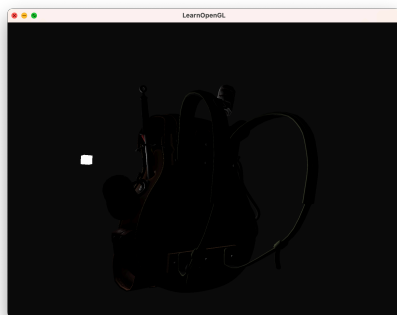


图 9: 实验 3 模式 3 背面结果

5 题外话

麻烦如果任务要求有更新的话请在群里发一下有更新的信息。改 model.h 改了两天最后告诉我不需要改可以直接改着色器真的很令人崩溃好吗好的谢谢。